Data Conflicts-Guided Interleaved Thread Scheduling for Flaky Test Detection in Multithreaded Programs

1st Tianzi Wang
dept. College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, P.R.China
2022200818@mail.buct.edu.cn

2rd Ruilian Zhao
dept. College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, P.R.China
rlzhao@mail.buct.edu.cn

3rd Weiwei Wang
dept. College of Information Engineering
Beijing Institute of Petrochemical Technology
Beijing, P.R.China
wangweiwei@bipt.edu.cn

4th Weixi Zhang
dept. College of Information Science and Technology
Beijing University of Chemical Technology
Beijing, P.R.China
https://orcid.org/0009-0001-5388-9478

2024 31st Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3315-3401-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/APSEC65559.2024.00021

***Abstract*—Flaky tests can non-deterministically pass or fail on the same version of code. Their occurrence prevents using test results to determine if there are bugs in the program, thereby reducing the credibility of software testing. Concurrency is a main cause of flaky tests, where different concurrent thread interleaving may lead to uncertain program execution results. Essentially, inconsistent test execution results are more likely to occur when threads with data dependencies are interleaved. But the existing work scheduled threads randomly with no guide during flaky test detection, resulting in a vast interleaving space and low efficiency. To address this issue, we propose a data conflict-guided flaky test detection approach, which prioritizes threads with data dependency for scheduling to try to alter the test results, thereby detecting flaky tests more effectively. In more detail, the dynamic execution trace of test cases is tracked in multithreaded programs. Then, by analyzing read-write operations in the trace, data conflicts are identified. On this basis, a Bayesian Network is put forward to evaluate the degree of impact of data conflicts on test flakiness, which implies potential flaky risk. That is, the greater the flaky risk, the more likely the data conflict is to cause a flaky test. So, the scheduling strategy prioritizes threads with data conflict pairs that have a significant impact on test flakiness, to trigger test flakiness as soon as possible. To verify the effectiveness of our approach, we conduct experiments on multithreaded Java programs. The experimental results show that our approach can discover the flakiness of tests within an average of 118 seconds, and the accuracy in detecting flaky tests can reach 75%.**

***Index Terms*—Flaky test, data conflict, multithreaded programs**

I. Introduction

Software testing provides an effective way to guarantee the quality and reliability of the software. However, flaky tests [1] damage the accuracy of testing, which undermines the assumption that a test failure necessarily indicates a bug, making software testing incredible. A flaky test occurs if the same test can be passed or failed even when the program being tested has not changed. A recent survey [2] revealed that 59% of testers encountered flaky tests monthly, weekly, or even daily, which consumes a significant amount of debugging time and effort. Therefore, how to detect flaky tests has been a critical issue.

Recent research [3] demonstrates that concurrency characteristics of programs are the main cause of flaky tests. As threads are the basic unit of concurrent execution, the flaky behaviors are always caused by complex thread interleaving or non-stable execution environments [4]. When multiple threads execute concurrently, the order in which their operations occur can vary with each run, resulting in different interleaving. The varying interleaving can expose subtle order-related bugs or race conditions that do not always manifest, causing tests to pass or fail unpredictably. Additionally, non-stable execution environments can exacerbate this issue by altering the timing of thread execution and resource contention. Consequently, the flaky behaviors make it challenging to consistently reproduce test results, leading to flaky tests.

As different thread interleaving may cause diverse test execution results, previous research has focused on actively controlling threads in the scheduling space to trigger flaky tests [5]. For example, Zhen et al. [6] proposed a flaky test detection method called FlakyScanner to identify flaky tests related to concurrency in Android applications by altering event execution sequences. By controlling the application execution in Android debug mode, suspending threads where asynchronous events occur, and monitoring test thread pauses waiting for test case event statements, FlakyScanner detected flaky tests in Android. Similarly, AATT [7] adapted GreenDroid [8] to guide dynamic analysis with thread scheduling, assisting in identifying flaky tests. Atomfuzzer [9] strategically

2640-0715/24/$31.00 ©2024 IEEE
DOI 10.1109/APSEC65559.2024.00021

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

manipulated execution schedules to provoke specific patterns, often necessitating numerous random schedules to uncover a flaky test caused by a bug. Deadlockfuzzer [10] employed a randomized thread scheduler to generate potential deadlocks with a high probability, which consequently resulted in flaky tests. However, these tools for concurrent programs are very time-consuming, and due to the lack of effective guidance, their scheduling space is excessively large. As a result, while these methods can detect flaky tests to some extent, optimizing scheduling guidance strategies is necessary.

In multi-threaded programs, data conflicts [11] may occur in the interweaving of multiple threads. Data conflict arises when multiple threads attempt to read and write shared data simultaneously without proper synchronization. These conflicts can cause race conditions, where the final state of the shared data depends on the unpredictable order of thread execution. In deadlock scenarios [12], each process is waiting for resources that are blocked and held by other processes. This results in a cyclic dependency where no process can proceed, leading to a system freeze. Tests involving potential deadlocks may intermittently pass or fail depending on the timing and order of resource requests, causing inconsistent test results and contributing to test flakiness. Consequently, these data conflicts and resulting race conditions introduce non-determinism, which greatly influences flaky tests. Therefore, to trigger flaky tests, it is crucial to identify the points where potential data conflicts may occur and explore flaky tests by altering the thread execution order based on these conflicts.

Thread Sharing Analysis (TSA) [13] provides a way to analyze the data conflict between threads. Most TSA technologies use Escape Analysis method, but it may miss many shared array accesses [14] and cause data conflict analysis results to be inaccurate. To address this issue, *Small Scope Hypothesis (SSH)* [15] is raised to use two-thread interleaving to trigger most concurrency issues. That is, if the shared data accesses between threads is within a small range of thread interleaving, it is more likely to cause concurrency issues. Inspired by the *small scope hypothesis*, this paper attempts to seek data conflicts within a small range of thread interleavings, in order to trigger as many flaky tests as possible.

Therefore, in this paper, a data conflict-guided flaky test detection approach is proposed, which identifies a small range of threads interleaving with data conflict by SSH and TSA and prioritizes them for scheduling to try to alter the test results, thereby detecting flaky tests more effectively. In more detail, the dynamic execution trace of test cases is tracked in multithreaded programs. Then, by analyzing read-write operations in a small range of threads interleaving in trace by TSA and SSH, data conflicts are identified. On this basis, a Bayesian Network is put forward to evaluate the degree of impact of data conflicts on test flakiness, which implies potential flaky risk. That is, the greater the flaky risk, the more likely the data conflict is to cause a flaky test. So, the scheduling strategy prioritizes threads with data conflict pairs that have a significant impact on test flakiness, to trigger test flakiness as soon as possible.

The main contributions of this paper are as below:

- For Java multithreaded programs, the trace of test cases in programs is tracked, and its access to shared data and threads interleaving is recorded and analyzed by TSA and SSH to identify data conflicts.
- A Bayesian Network is put forward to measure the influence degree of data conflict pairs on flaky test. And a scheduling strategy is designed to prioritizes threads with data conflict pairs that have a significant impact on test flakiness, to trigger test flakiness as soon as possible.
- To verify the effectiveness of our approach, we conduct experiments on multithreaded Java programs. The experimental results show that our approach can discover the flakiness of tests within an average of 118 seconds, and the accuracy in detecting flaky tests can reach 75%.

The remaining of the paper is organized as follows: Section 2 presents a motivating example to illustrate the main goal of our approach. Section 3 describes our method framework, detailing the specific approaches and techniques employed. Section 4 reports our implementation details, the experiment settings, and evaluation results. Section 5 concludes the paper and presents the threats to validity.

II. MOTIVATING EXAMPLE

In multithreaded programs, the thread execution order of a program can significantly impact the final outcome. For instance, consider Listing 1, a seat sales system involving two critical methods, *sell()* and *cancelSale()*, along with shared data *numberOfSeatsSold* (tracking the number of seats sold) and *stopSales* (controlling whether sales should stop). The method *sell()* increases the count of sold seats and potentially stops further sales if the maximum capacity is reached or exceeded. The method *cancelSale()* decreases the count of sold seats and adjusts the sales availability flag based on the current seat count and maximum capacity conditions. In the example of Listing 2, our test case evaluates the functionality of the seat sales system.

There are two threads T_1 and T_2: T_1 executes the *sell()*, and T_2 executes *cancelSale()*. Under normal circumstances, the test should be able to verify the functionality of increasing or decreasing the value of *numberOfSeatsSold* correctly by alternately executing threads T_1 and T_2 within the *sell()* and *cancelSale()* methods. Additionally, it should properly manage the $stopSales$ to ensure system consistency. However, under specific thread interleaving, it may result in flaky tests. The test thread T_1 initiates the *sell()* method, first verifying if *numberOfSeatsSold* is within *maximumCapacity* (line 2). Upon meeting this condition, T_1 increments *numberOfSeatsSold* (line 3). After if *numberOfSeatsSold* equals *maximumCapacity*, T_1 checks whether *stopSales* is false. If *stopSales* is false, T_1 sets *stopSales* to true (line 6) to halt further sales operations. If *stopSales* is already true, indicating sales have stopped, the thread decrements *numberOfSeatsSold* once (line 8) to prevent exceeding the *maximumCapacity*. If between checking *stopSales* and executing the corresponding logic (lines 5 to

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

```
1  public void sell() {
2      if (numberOfSeatsSold <= maximumCapacity) {
3          numberOfSeatsSold++;
4          if (numberOfSeatsSold >= maximumCapacity) {
5              if (!stopSales) {
6                  stopSales = true;
7              } else {
8                  numberOfSeatsSold--;
9              }
10         }
11     }
12 }
13
14 public void cancelSale() {
15     if (numberOfSeatsSold > 0) {
16         numberOfSeatsSold--;
17         if (stopSales && numberOfSeatsSold <
               maximumCapacity) {
18             stopSales = false;
19         }
20     }
21 }
```

Listing 1. An example of managing seat sales

```
1 TestSellCancelSale test = new TestSellCancelSale();
2 Thread T1 = new Thread(test::sell);
3 Thread T2 = new Thread(test::cancelSale);
4 T1.start();
5 T2.start();
```

Listing 2. The test case

8), another test thread T_2 successfully decreases *numberOfSeatsSold* (line 16), and sets *stopSales* to false (line 18), T_1 may incorrectly continue to increase *numberOfSeatsSold* (line 3), potentially exceeding the *maximumCapacity*. From the comparison of the above two scenarios, it is evident that although the total order of statements in the test case does not change, shared data conflicts occur due to different thread interleaving, resulting in a flaky test. For example, in the seat sales management scenario, it is precisely the different access sequences to the *numberOfSeatsSold* and *stopSales* variables can result in ticket sales errors. This unstable access sequence of shared data directly or indirectly leads to a flaky test. To detect the flakiness of test cases, we need to control thread scheduling to trigger data conflicts, thereby identifying flaky test cases and enhancing the credibility of software testing.

III. METHOD

To detect the flakiness of a test case in a multithreaded program, we control thread interleaving to produce different outcomes in the test. Since thread interleaving with data dependencies to cause data conflicts is more likely to alter test results, we propose a method for detecting test flakiness by guiding thread scheduling based on read-write data conflicts, as illustrated in Fig. 1.

Our approach is divided into three parts as follows: 1. Conflict pair generation based on TSA and SSH. To obtain data dependencies between threads, we collect information on threads, variables, and lock sets during test execution to construct a dynamic execution trace. We use TSA and SSH to identify read-write data conflicts related to variables between threads, and generate data conflict pairs. 2. Priority evaluation of conflict pairs based on Bayesian Analysis. In order to identify and prioritize conflict pairs that have the greatest impact on test flakiness, we evaluate the conflict pairs. Based on the influence factors analysis of test flakiness, We define a metric called "flaky degree" to assess this impact, and build a Bayesian network to evaluate the conflict pairs based on the evaluation indicators of the impact factors of the conflict pairs on the flaky test. Ultimately, we obtain a set of conflict pairs sorted by their flaky degree. 3. Thread scheduling targeting conflict pairs. Based on the priority results of conflict pairs, we sequentially select a conflict pair to guide thread scheduling. Finally, we can judge whether the guidance of the conflict pair is effective according to the assertion result of the test.

A. Conflict Pair Generation Based on TSA and SSH

1) The definition of a test case execution trace: As previously mentioned, we need to collect information on threads, variables, and lock sets to identify read-write data conflicts related to variables between threads and construct data conflict pairs to guide subsequent thread scheduling and trigger test flakiness. Where lock set information is used to filter out infeasible thread schedules, that means excluding data conflict pairs that cannot guide thread scheduling. Therefore, we record operations involving variables, threads, and lock sets during test execution and classify them into three types based on the operations, as shown in Table I.

TABLE I
THE TYPE OF OPERATION IN TRACE

Category	Operation	Description
Thread	*FORK(u,v)*	Thread u forks thread v
	JOIN(u,v)	Thread u waits for thread v to complete
	STOP(u)	Thread u stopped
Lock	*ACQ(u,l)*	Thread u acquires lock l
	REL(u,l)	Thread u releases lock l
Variable	*R(u,x)*	Thread u reads x
	W(u,x)	Thread u writes x

For thread operations, there are three situations: *FORK*, *JOIN*, and *STOP*. Lock set protection has two types: *ACQ* and *REL*, corresponding to thread acquiring or releasing a lock (we only consider mutex locks). For variable operations, there are two cases: read and write, represented by R and W, respectively.

Our trace is composed in three distinct forms based on three different types of operations in Table I:

- O_i^T = *(FORK/JOIN/STOP,* $T\{T_a, T_b\}$*)*
- O_i^L = *(ACQ/REL, T, L)*
- O_i^V = *(R/W, T, x, La)*

For thread operations, T represents thread. O represents operation, and O_i^T refers to the i-th thread operation. In the interaction between two threads, O_i^T = *(FORK,* $T\{T_a, T_b\}$*)* indicates that T_a forks a task to T_b. O_i^T = *(JOIN,* $T\{T_a, T_b\}$*)* indicates that T_a blocks the current thread and waits for T_b to complete. O_i^T = *(STOP,* $T\{T_a\}$*)* means that T_a has been paused. Concerning the operations on lock sets, there are two types: O_i^L = *(ACQ/REL, T, L)*, which respectively denote that thread T acquires or releases a lock L. In this sequence of

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

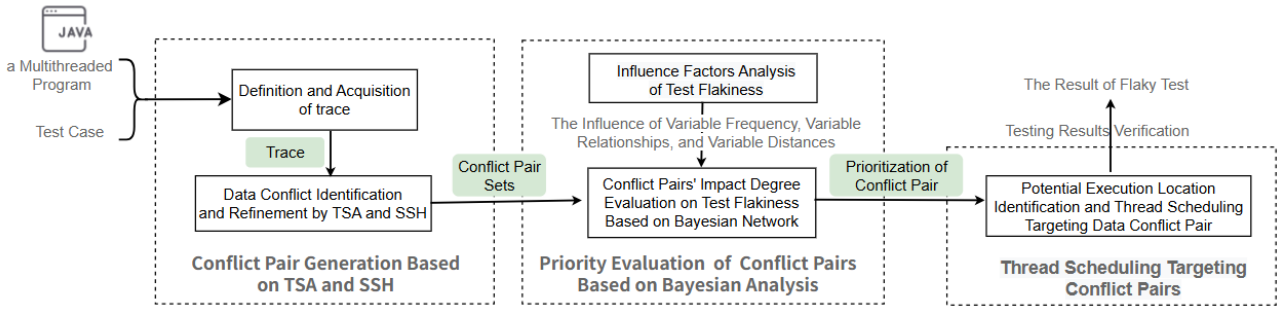


Fig. 1. System Framework Diagram

operations trace, the definition of variable access situations is shown as O_i^V. Here, *R/W* indicates read or write operation, *T* represents the current operating thread, *x* denotes the variable being accessed in the current operation, and *La* stands for the program point used to mark different accesses. The program point is the location in the trace.

Therefore, our trace takes the form: Trace = $\langle O_1^T, O_2^V, O_3^L, ..., O_i^V, ..., O_j^V, ... \rangle$. Note that this trace is just a formal example. In practice, the order of O_i^T, O_i^V, and O_i^L is determined by the execution.

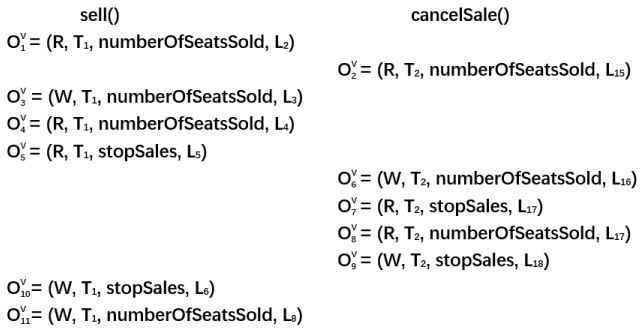


Fig. 2. A test case execution scenario

2) Data conflict pair generation: Current research has shown the effectiveness of the SSH, indicating that for multithreaded programs, a small number (2 times) of thread interleaving can trigger the majority of flaky tests. Therefore, we focus on data conflicts between two threads, with the aim of altering the test execution results by scheduling these two threads. Thus, it is necessary to identify conflicts between variable operations on these two threads and construct conflict pairs. Our definition of conflict pairs is as follows in Definition 1.

Definition 1. *Conflict pairs: If two threads perform different operations on a shared variable, and at least one of these operations is a write operation, these operations have data conflict relationships. A conflict pair consists of several O_i^V with data conflict relationships. Their order in the conflict pair is different from the original trace, creating potential data conflicts.*

We use TSA to analyze data conflicts between threads. Specifically, using the three types of operations we defined earlier in the trace, conflict pairs are generated based on variable information in the traces. Conflict pairs describe how threads access one-or-two shared variables. $\langle O_1^V, O_2^V \rangle$ and $\langle O_1^V, O_2^V, O_3^V \rangle$ represent single-variable conflicts, specifically cases of two read-write accesses and three read-write accesses. $\langle O_1^V, O_2^V, O_3^V, O_4^V \rangle$ represents double-variable conflicts of four read-write accesses, as shown in Table II. Here O_i^V is a representation of the variable type of operation we defined earlier. To introduce the generation of conflict pairs, the example in Fig. 2 illustrates a thread execution scenario involving only variable operations. A conflict pair, $\langle O_8^V, O_{11}^V \rangle$, regarding the variable *numberOfSeatsSold*, can be constructed. This pair is represented as $\langle (R,\ T_2,\ numberOfSeatsSold,\ L_{17}),\ (W,\ T_1,\ numberOfSeatsSold,\ L_8) \rangle$. Based on the definition of conflict pairs, we introduce our conflict pair constraint generation algorithm and the constraints on conflict pairs.

TABLE III
CONSTRAINTS TO CONFLICT PAIRS

No	Constraint	Description (the conflict pairs in ConflictSet)
1	No Redundancy	Guarantee no redundant
2	Lock Safety	Violate the correct lock set protection cannot be generated
3	Same Modes	Guarantee generated within the different modes

Constraints on conflict pairs. Redundancy and invalid pairs are inevitable during the generation of conflict pairs. In our conflict pair generation method, all possible conflict pairs are generated through combinations based on two read-write access situations. This combinatorial approach leads to the possibility of redundancy. Due to lockset protection and other mechanisms, not all thread schedules based on conflict pairs can be executed. To ensure that conflict pairs generated can effectively guide thread scheduling, we need to filter out some conflict pairs. Only schedulable conflict pairs in different forms can be kept. Constraints are as follows: No Redundancy: Ensures that the generated conflict pairs are free from redundancy. In other words, there should be no duplicate or redundant conflict pairs within the conflict set, ensuring each conflict pair is unique and necessary. This helps reduce the size of the conflict set and avoids unnecessary computational overhead. Lock Safety: Ensures that no conflict pairs violate correct lockset protection. If a conflict pair that violates lockset protection is generated, it will not be implemented during scheduling as it contradicts the lock safety. Same Modes: conflict pairs should be generated under different modes. "Modes" refers to the particular ways or processes in which a program executes certain operations, such as loops, parallel

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THREE TYPES OF READ-WRITE ACCESS CONFLICTS

No.	Description	Conflict pairs
1	Two read-write accesses of a single variable $\langle O_1^V, O_2^V \rangle$	$(R, T_a, x, La)(W, T_b, x, Lb)$
2		$(W, T_a, x, La)(R, T_b, x, Lb)$
3		$(W, T_a, x, La)(W, T_b, x, Lb)$
4	Three read-write accesses of a single variable $\langle O_1^V, O_2^V, O_3^V \rangle$	$(R, T_a, x, La)(W, T_b, x, Lb)(R, T_a, x, Lc)$
5		$(W, T_a, x, La)(W, T_b, x, Lb)(R, T_a, x, Lc)$
6		$(W, T_a, x, La)(R, T_b, x, Lb)(W, T_a, x, Lc)$
7		$(R, T_a, x, La)(W, T_b, x, Lb)(W, T_a, x, Lc)$
8		$(W, T_a, x, La)(W, T_b, x, Lb)(W, T_a, x, Lc)$
9	Four read-write accesses of two variables $\langle O_1^V, O_2^V, O_3^V, O_4^V \rangle$	$(W, T_a, x, La)(W, T_b, x, Lb)(W, T_b, y, Lc)(W, T_a, y, Ld)$
10		$(W, T_a, x, La)(W, T_b, y, Lb)(W, T_b, x, Lc)(W, T_a, y, Ld)$
11		$(W, T_a, x, La)(W, T_b, y, Lb)(W, T_a, y, Lc)(W, T_b, x, Ld)$
12		$(W, T_a, x, La)(R, T_b, x, Lb)(R, T_b, y, Lc)(W, T_a, y, Ld)$
13		$(W, T_a, x, La)(R, T_b, y, Lb)(R, T_b, x, Lc)(W, T_a, y, Ld)$
14		$(R, T_a, x, La)(W, T_b, x, Lb)(W, T_b, y, Lc)(R, T_a, y, Ld)$
15		$(R, T_a, x, La)(W, T_b, y, Lb)(W, T_b, x, Lc)(R, T_a, y, Ld)$
16		$(R, T_a, x, La)(W, T_b, y, Lb)(R, T_a, y, Lc)(W, T_b, x, Ld)$
17		$(W, T_a, x, La)(R, T_b, y, Lb)(W, T_a, y, Lc)(R, T_b, x, Ld)$

processing, or specific algorithms. In these modes, there are often many conflict pairs, but they are generated by a single mode. For instance, in a loop mode with n iterations, there could be a multiple of n-conflict pairs under the same mode. These n-conflict pairs reflect data conflicts in the same mode. These conflicts have different access points in trace, but the actual block of code that reacts at execution time remains the same. In this case, we need to generate as many conflict pairs in different modes as possible, rather than generating many conflict pairs reflecting the same mode. The Same Modes constraint eliminates this impact. By introducing these constraints, the number and quality of conflict pairs can be effectively controlled. This ensures that the generated conflict pairs accurately reflect potential data conflicts in program execution, thereby guiding thread scheduling.

Conflict pair constraint generation algorithm. In this part, we introduce our conflict pair generation algorithm. The input is the execution traces of test cases, and the output is a set of conflict pairs that exhibit data conflicts. To generate conflict pairs as shown in Table II, the algorithm first identifies shared variables in the current program execution trace. The algorithm constructs two-access conflict pairs $\langle O_1^V, O_2^V \rangle$ based on the read-write information (lines 8-14). The two-access conflict pair means two read-write accesses of a single variable in Table II. The three-access conflict pair and the four-access conflict pair can be derived by combining these basic conflict pairs $\langle O_1^V, O_2^V \rangle$. The three-access conflict pair means three read-write accesses of a single variable while four-access conflict pair means four read-write accesses of two variables. The var in the algorithm represents the specific variable. That is, the shared variables involved in the conflict in the program. Within the conflict set, for each existing two-access conflict pair c_1, the algorithm searches for another two-access conflict pair c_2 where "$c_2.O_1^V$ == $\mathrm{c}_1.O_2^V$", indicating that $\mathrm{c}_2.O_2^V$ should be the third operation, forming a new conflict pair $\langle O_1^V, O_2^V, O_3^V \rangle$, which is then added to the conflict set *ConflictSet* (lines 15-21). For generating four-access conflict pairs, specifically in the form of $\langle O_1^V, O_2^V, O_3^V, O_4^V \rangle$, the algorithm identifies two-access pair c_1, and searches for another two-access pair c_2 where "$c_2.O_1^V$ == $\mathrm{c}_1.O_2^V$", ensuring that the thread accessing satisfies the specific requirements for four read-write accesses as shown in Table II. This process forms a new conflict pair $\langle O_1^V, O_2^V, O_3^V, O_4^V \rangle$, which are then added to the conflict set *ConflictSet* (lines 22-30).

The constraints on conflict pairs are achieved through the *satisfies_constraints* function, which imposes constraints on conflict pairs based on the conditions specified in Table III.

B. Analysis of Flaky Degree in Conflict Pairs Based on the Bayesian Network

To analyze the impact of different conflict pairs on the flaky degree of the current test cases, we use a Bayesian Network to access the flaky degree of conflict pairs. We first need to identify the factors influencing the flaky degree. Frequent variable access by multiple threads increases conflict likelihood, variable interactions can intensify sensitive data dependencies, and the distance of program points (i.e., La in O_i^V) between the conflict pair affects the strength and timing of these interactions. So our Bayesian Network model includes: Analysis of Variable Access Frequency Impact (VAF): This focuses on how frequently variables are accessed by multiple threads and its impact on conflict conditions. Analysis of Variable Interactions (VI): This examines the dependencies between variables and how their interactions can lead to flaky behaviors. Program Point Span Impact Analysis (PSA): This assesses the impact of the distance between program points of a conflict pair on the strength of variable interactions. In a conflict pair, program points are the location of variables in the trace.

Analysis of Variable Access Frequency Impact (*VAF*). The analysis of VAF can be divided into single-variable access situations and double-variable access situations. In the single-variable situations, only the access frequency of a single variable (x as an example) is considered, which is the number of conflict pairs involving variable x in the conflict set. In

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

O_1^v = (W, T_2, x, La)
O_2^v = (R, T_1, y, Lb)
O_3^v = (R, T_1, y, Lc)
O_4^v = (R, T_4, x, Ld)
O_5^v = (W, T_3, y, Le)
O_6^v = (W, T_1, x, Lf)
O_7^v = (R, T_2, y, Lg)
O_8^v = (W, T_2, y, Lh)

Fig. 3. Simplified Execution Trace Sequence Example

the double-variable situations, the access frequency of two variables (x and y as an example) is considered, which is the number of conflict pairs involving variables x and y in the conflict set, excluding duplicates. This approach allows for more effective identification and classification of the access priorities of different variables.

The priority of VAF is calculated using (1), where n is the number of variables determined based on the type of conflicting pair (single-variable or double-variable). In the single-variable situation, it is the number of conflicting pairs involving a single variable; in the double-variable situation, it is the number of conflict pairs where two variables co-occur. β_0 and β_1 are the logistic regression parameters used to predict the probability of *VAF* occurring for different values of n. Specifically, β_0 adjusts the baseline probability of VAF occurring when there are no conflict pairs ($n = 0$). On the other hand, β_1 is the coefficient that determines how quickly the probability of VAF increases as the number of conflicting pairs n grows. A positive β_1 would indicate that as n increases, the likelihood of VAF occurrence rises.

$$P(VAF) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 n)}} \tag{1}$$

Analysis of Variable Interactions (*VI*). When multiple read-write interactions occur within conflict pairs, they inevitably affect the other read-write accesses within the program access points range of the conflict pair. Consider the example of single-variable interactions involving two read-write accesses and double-variable interactions involving four read-write accesses, as illustrated in Fig. 3, which represents a simplified execution trace sequence. For single-variable conflict pair $\langle O_2^V, O_8^V \rangle$, O_2^V and O_8^V affect variable y through read and write operations, respectively. When analyzing these read-write interactions, it is also necessary to consider other read-write impacts on y (excluding the effects on x if the conflict pair only involves y). Indirectly, there are read-write impacts from O_3^V, O_5^V, and O_7^V. The read operations can get the variable's value without modifying it, while only the write operations O_5^V affect the current conflict pair $\langle O_2^V, O_8^V \rangle$. Similarly, for double-variable conflict pair $\langle O_1^V, O_2^V, O_6^V, O_7^V \rangle$, O_5^V also exerts a write impact. This influence is included in the impact analysis of variable interactions.

$$P(VI) = \frac{1}{1 + e^{-(\alpha(c+d) + \beta e)}} \tag{2}$$

$$e = \frac{|Ux \cap Uy|}{|Ux \cup Uy|} \tag{3}$$

Although VI does not directly map to the complexity of dependency chains, it indirectly reflects the potential complexity of variable interactions. The core idea here is that the number of interactions between variables can serve as an indirect indicator of the degree of interaction between variables in the current conflict pair and other variables. An increase in variable interactions implies a higher chance of triggering flaky tests in concurrent execution contexts. Each interaction point could become a trigger for flaky tests. Therefore, the greater the number of interactions between variables, the higher the potential risk of an overall flaky test.

To quantify this effect, (2) can be used: c and d respectively refer to the number of interactions with other variables in the conflicting pairs involving variable x and variable y; e is the overlap ratio between variable x and variable y (i.e., the impact of x and y interacting with other variables together), calculated as shown in (3). In the case of a single-variable situation, e is not applicable.

$$Pspan = \left(\frac{Cou}{Dis^{\alpha} + 1} \right) \times (K - H(Dis - L)) \tag{4}$$

$$P(PSA) = \frac{1}{1 + e^{-Pspan}} \tag{5}$$

Program Point Span Impact Analysis (*PSA*). PSA aims to determine the correlation of conflict set by assessing the distance between program access points. Program access points that are closer in proximity typically indicate that they are part of the same or similar program logic, hence having a stronger correlation. Conversely, access points that are farther apart often belong to different program logic parts and thus have a weaker correlation. To distinguish between "near" and "far" program points, a threshold can be set. Within this threshold, program points with shorter distances and frequent interactions are considered to have a high correlation, while those with longer distances and sparse interactions outside the threshold are considered to have a low correlation.

The calculation formulas for PSA are shown in (4) and (5). *Dis* represents the distance between the program access points of two variables, it actually refers to the number of operations between two variables in the trace. *Cou* represents the number of shared variable accesses between the considered pairs of program points (i.e., the frequency of interaction). α is a positive number used to adjust the impact of distance on priority. When *Dis* is large, the increase of Dis^{α} will slow down the growth of priority, reflecting the decrease in relevance with longer distances. K is a positive coefficient used to adjust the absolute value of the priority. *H(x)* is a step function: when *Dis* $\geq L$, *H(Dis-L) = H*, reflecting the decrease in priority when the distance exceeds a certain limit; when *Dis* $<L$, *H(Dis-L) = 0*, it does not affect the priority.

As shown in the Bayesian Network in Fig. 4, there are 10 nodes. If a *Conditional Probability Table (CPT)* were created for these 10 nodes, it would contain 2^{10} (1024) rows to cover all possible state combinations of the 10 nodes. CPT specifies the probability distribution for a node given its parent nodes in a Bayesian Network. In a Bayesian Network, traditional CPTs can be replaced by functions, with the probability of each

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

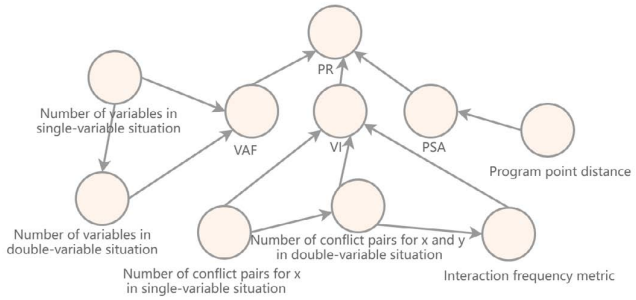


Fig. 4. Conflict pair assessment based on the Bayesian Network

node being calculated by a well-defined set of mathematical formulas based on the states of other nodes in the network. We consider a modular and parameterized approach to simplify CPTs, using mathematical models (such as context-specific formulas) to define probabilities instead of manually specifying each entry. By decomposing the network into smaller, interrelated modules, we can perform quantitative modeling of the Bayesian Network.

C. Thread Scheduling Targeting Conflict Pairs

After calculating the flaky degree of conflict pairs using the Bayesian Network, we select the conflict pair with the highest flaky degree to guide thread scheduling. As stated in Definition 1, the order of operations in conflict pairs can directly lead to data conflicts. Therefore, our conflict pairs can directly guide the scheduling of threads. Since our conflict pairs only involve read-write and write-write situations, there will be no read-read conflict pairs as read-read situations do not cause data conflicts. We categorize into the following two situations:

- Read-Write (*RW/WR*) situation: The conflict pair includes both read and write operations. During scheduling, it is necessary to consider the impact of read and write operations. Our goal is to ensure that read operations are not affected by other threads when scheduling between the current two threads.
- Write-Write (*WW*) situation: The conflict pair does not include read operations, only write operations. We ensure that the scheduling of the current two threads is affected by write operations.

Based on the situations above, we focus on the scheduling of the conflict pair we selected only between two threads(because of SSH), while keeping the execution order of other threads as close to the original test execution trace as possible. If we directly swap the execution order according to the conflict pair, we cannot ensure that other threads remain unaffected. Therefore, we use a thread scheduling strategy guided by the conflict pair we selected as follows: 1. When the conflict pair is a Read-Write situation, (1) for single-variable conflicts (x as an example), we protect the read operation from being affected by other threads when scheduling between the current two threads. We analyze whether there are any other threads affecting the conflict pair on the trace. 1⟩ If there are no other threads affecting it, we directly swap the operation order by the order of the conflict pair. 2⟩ If other threads do affect it and the effect is a read operation on x, we directly swap the operation order by the order of the conflict pair; 3⟩ If the effect is a write operation on x, the read operation in the current conflict pair remains unchanged. The write operation in the conflict pair is moved before or after the read operation to ensure that other threads do not affect the read of x. (2) For double-variable conflicts (x and y as an example), the method is essentially the same as for single-variable conflicts.

Algorithm 1 ConflictPairsConstraintGeneration

Input: variables V in trace
Output: ConflictSet

function CONFLICTPAIRSCONSTRAINTGENERATION(Trace, ConflictSet)
  ConflictSet $\leftarrow \emptyset$
  for all variable x in V **do**
    if x is not shared variable **then**
      remove x in V
    end if
  end for
  for all variable x in V **do**
    for all O_1^V and O_2^V in Trace **do**
      if $O_1^V \neq O_2^V$ and $O_1^V.var == O_2^V.var$ and $O_1^V.thread \neq O_2^V.thread$ **then**
        ConflictSet.add($\langle O_1^V, O_2^V \rangle$)
      end if
    end for
  end for
  for all Conflict c in ConflictSet and $len(c) == 2$ **do**
    for all c_1 and c_2 in ConflictSet **do**
      if $c_1 \neq c_2$ and $c_2.O_1^V == c_1.O_2^V$ **then**
        ConflictSet.add($\langle c_1.O_1^V, c_1.O_2^V, c_2.O_2^V \rangle$)
      end if
    end for
  end for
  for all Conflict c in ConflictSet and $len(c) == 2$ **do**
    for all c_1 and c_2 in ConflictSet **do**
      if $c_1 \neq c_2$ and $c_2.O_1^V == c_1.O_2^V$ and $c_1.O_1^V.thread == c_2.O_2^V.thread$ and $c_1.O_2^V.thread == c_2.O_1^V.thread$ **then**
        ConflictSet.add($\langle c_1.O_1^V, c_1.O_2^V, c_2.O_1^V, c_2.O_2^V \rangle$)
        ConflictSet.add($\langle c_1.O_1^V, c_2.O_1^V, c_1.O_2^V, c_2.O_2^V \rangle$)
        ConflictSet.add($\langle c_1.O_1^V, c_2.O_1^V, c_2.O_2^V, c_1.O_2^V \rangle$)
      end if
    end for
  end for
  for all Conflict c in ConflictSet **do**
    if not satisfies_constraints(c) **then**
      remove c from ConflictSet
    end if
  end for
  return ConflictSet
end function

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

However, when the effect is write operations on both x and y, we need to protect the reads of both x and y. We calculate the divergence between the new scheduling sequence and the original trace for both scenarios: protecting the read operation of x and protecting the read operation of y. We then select the variable with the smaller divergence to protect its read operation, thereby minimizing the impact on other threads. 2. When the conflict pair is a Write-Write situation, we swap the operation order directly followed by the order of the conflict pair.

Specifically, assuming the execution trace is as shown in Fig. 3, here are some examples of scheduling situations: Such as $\langle O_1^V, O_4^V \rangle \rightarrow \langle O_4^V, O_1^V \rangle$ and $\langle O_5^V, O_7^V \rangle \rightarrow \langle O_7^V, O_5^V \rangle$, when there are no other operations influencing the conflict pair in trace, directly swap the order. For single-variable conflict pair: $\langle O_5^V, O_7^V \rangle \rightarrow \langle O_7^V, O_5^V \rangle$. There is a write operation O_6^V on x between the conflict pair in trace, prioritize maintaining the read operation O_7^V. This results is moving operation O_5^V after O_7^V. The idea is to ensure that the result of the read operation in the conflict pair is not affected by other threads. If the conflict pair consists entirely of write operations, such as $\langle O_1^V, O_6^V \rangle \rightarrow \langle O_6^V, O_1^V \rangle$, we ensure the scheduling of current two threads is affected by write operations by swapping the order of O_1^V and O_6^V. For two-variable conflict pair: $\langle O_1^V, O_2^V, O_6^V, O_7^V \rangle \rightarrow \langle O_7^V, O_1^V, O_6^V, O_2^V \rangle$. There are two read operations O_2^V and O_7^V, and we prioritize maintaining the order of the read operation O_2^V, the execution sequence becomes O_7^V, O_1^V, O_6^V, O_2^V, O_3^V, O_4^V, O_5^V, O_8^V (O_2^V remains unchanged, O_7^V and O_6^V are moved earlier). If we prioritize maintaining the order of the read operation O_7^V, the execution sequence becomes O_3^V, O_4^V, O_5^V, O_7^V, O_1^V, O_6^V, O_2^V, O_8^V (O_7^V remains unchanged, O_1^V, O_2^V, and O_6^V are moved later). In such cases, we need to calculate the sequence differences to find the scheduling with the least impact.

To optimize the scheduling strategy for conflict pairs, we consider the following three points: Comprehensive evaluation of the impact between conflict pairs. By considering the read-write access situations and control dependencies between conflict pairs, we ensure the comprehensiveness and accuracy of the scheduling strategy. Calculation of scheduling sequence differences. We calculate the differences between scheduling sequences and original sequences to quantify the impact of different scheduling orders, allowing us to choose the order with the least impact. Elimination of the impact of read-write operations. We focus on the direct impact of write operations on read operations, ensuring that data changes read are caused only by the intended write operations. As for write-write operations, we aim to amplify the impact of write operations as much as possible to explore flaky tests.

IV. IMPLEMENTATION AND EVALUATION

In this section, we present our implementation details, the experiment settings, and evaluation results. To verify the effectiveness of the proposed read-write data conflict-based detection method for concurrency-related flaky tests, a series of experiments are designed and conducted on 7 test cases from 4 Java multithreaded programs to evaluate the effectiveness of the proposed method.

We use Thread Sharing Analysis technology to provide share analysis among threads and obtain trace. We optimize classic dynamic escape analysis and identify program locations in concurrent programs that access thread shared data and record them. Import the java multithreaded project to be tested into the IDE, and our dynamic sharing analysis libraries will analyze it. In Java multithreaded programs, the Instrumentor is a component used to dynamically modify bytecode at runtime for various purposes, including controlling thread scheduling. Based on Instrumentor, we implement a custom thread scheduler that overrides the default scheduling behavior of the Java Virtual Machine (JVM). Based on conflict pair we selected, we use ASM library to modify the Java bytecode of classes, making the bytecode file conform to our thread scheduling strategy. By controlling thread scheduling, they can help us check whether conflict pairs we selected trigger flaky tests.

Our evaluation aims to answer three questions:

RQ1: Can the data conflict analysis method based on Thread Sharing Analysis effectively detect flaky tests in test cases?

To address Research Question 1, we design a set of experiments to analyze the time overhead and the number of detections of shared read-write data conflicts. This is done to verify the effectiveness of the proposed method for detecting read-write data conflicts in test traces.

Based on the analysis of Table IV, it is observed that the number of conflict pairs does not necessarily increase with the increase in the length of trace and the number of shared variable read-writes. As we analyzed in Section 3, the constraints on conflict pairs control the number of feasible conflict pairs. Due to lockset protection and the inherent access order of threads, an exponential number of conflict pairs will not be generated during actual execution. Additionally, the read-write data conflict detection method proposed in this paper incurs very low time overhead when detecting read-write data conflicts in test case execution traces, averaging the completion of one test case detection every 1.9 seconds.

In Fig. 5, the results show our testing outcomes for the top 10% of prioritized conflict pairs. "*Flaky*" indicates the number of successfully detected flaky tests, while "*Not Flaky*" indicates those not successfully detected. Table V lists the test cases in the first column and the number of conflict pairs selected in the second column (selected based on the top 10% prioritized conflict pairs). The third column shows the number of flaky tests, and the detection accuracy is in the fourth column. Experimental results demonstrate that when using our proposed prioritization strategy guided by conflict pairs, the accuracy in detecting flaky tests reaches 75%. And our detection method can identify flakiness within an average of 118 seconds. This validates the effectiveness of our proposed method for detecting read-write data conflicts in test case execution traces and detecting flaky tests.

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DETECTION RESULTS BASED ON READ-WRITE DATA CONFLICTS

Test Case	Number of Shared Variables	Length of Trace	Number of Shared Variable Read-Writes	Number of Conflict Pairs	Time (s)
Producer-Consumer	3	254	99	171	1.6
ThreadTest	1	44	8	51	1.1
ThreadsMany	2	47	16	366	2.1
ThreadsFew	2	57	20	460	2.3
ThreadsFull	2	81	28	768	2.7
bufwriter1	3	367	49	198	1.4
bufwriter2	3	452	34	176	2.1
average					1.9

TABLE V
DETECTION RESULTS BASED ON CONFLICT PAIRS FOR FLAKY TESTS

Test Case	Number of Conflict Pairs Detected	Number of Flaky Tests	Accuracy (%)
Producer-Consumer	17	13	76.5
ThreadTest	5	4	80
ThreadsMany	37	26	70.3
ThreadsFew	46	40	87
ThreadsFull	77	57	74
bufwriter1	20	13	65
bufwriter2	18	13	72.2
average			75

RQ2: Can the Bayesian Network-based prioritization method for conflict pairs effectively guide the detection of flaky tests?

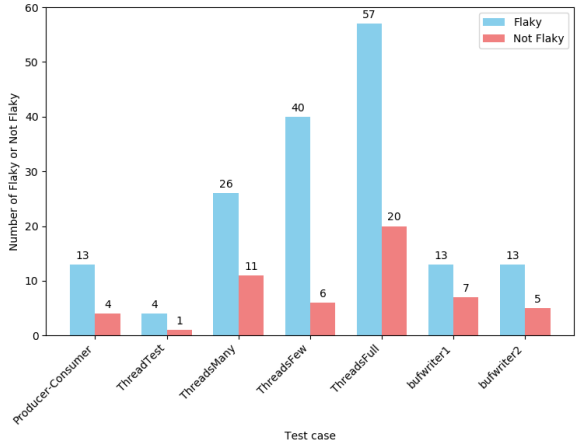
To effectively evaluate the Bayesian Network-based prioritization method for conflict pairs, we introduce the concept of "Average Detection Steps" (ADS). ADS measures the number of steps required to detect a flaky test. The fewer steps needed to trigger a flaky test, the more effective our prioritization method is. To validate this, we compare our method against the Random Test approach. As shown in Table VI, our method averages need 1.29 steps to trigger a flaky test. That is to say, according to our method of verifying conflict pairs, on average, a flaky test can be triggered by the second conflict pair. In contrast, the Random Test randomly selects 30 sets of 10% conflict pairs from the conflict set for each test case. The Random Test, on average, triggers a flaky test by the 12th conflict pair. Experimental results indicate that our method significantly outperforms the Random Test.

TABLE VI
COMPARISON OF ADS METRICS BETWEEN OUR METHOD AND RANDOM DETECTION

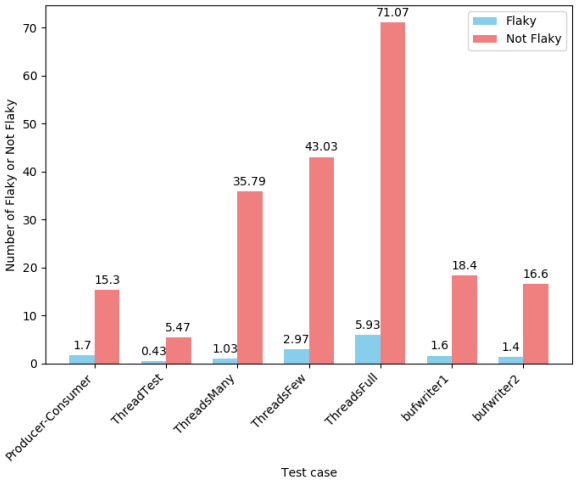
Test Case	Our Method Test	Random Test (Average)
Producer-Consumer	1	9.1
ThreadTest	2	2.3
ThreadsMany	1	13.8
ThreadsFew	1	15.1
ThreadsFull	1	18.9
bufwriter1	1	8.7
bufwriter2	2	9.5
avarage	1.29	11.06

RQ3: How does the effectiveness of the proposed flaky test detection method, based on shared data conflict, compare with the Random Test?

To assess the effectiveness of our method for detecting flaky tests compared to existing approaches, we conduct



(a) Our Method



(b) Random Test

Fig. 5. Conflict pair assessment based on the Bayesian Network

experiments comparing the Random Test with our Bayesian Network-based prioritization of conflict pairs. For each test case, our method prioritizes the top 10% of conflict pairs based on the result of the Bayesian Network, whereas the Random Test randomly selects 30 sets of 10% conflict pairs from the entire set. By comparing the experimental results from Table V and Table VII, our method consistently detects flaky tests more quickly on average for each test case. The results show that our method effectively detects the flaky test and 67.7% more

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
DETECTION RESULTS OF FLAKY TESTS IN RANDOM EXPERIMENTS

Test Case	Number of Conflict Pairs Detected	Number of Flaky Tests	Accuracy (%)
Producer-Consumer	17	1.7	10
ThreadTest	5	0.43	8.6
ThreadsMany	37	1.03	2.7
ThreadsFew	46	2.97	6.4
ThreadsFull	77	5.93	7.7
bufwriter1	20	1.6	8
bufwriter2	18	1.4	7.8
average			7.3

than the Random Test.

V. CONCLUSION AND THE THREATS TO VALIDITY

In this section, we conclude the paper and present the threats to validity. In complex multithreaded environments, read-write data conflicts occurring during test case execution are a major cause of concurrency-related flaky tests. By thoroughly analyzing these read-write data conflicts, we have designed a method to guide the scheduling of test execution traces, enabling more efficient detection of flaky tests. We propose a concurrency-related flaky test detection method based on data conflicts. The method first detects read-write data conflicts in the execution traces of test cases and abstracts conflict pairs. Then, it evaluates the flaky degree of these conflict pairs using a Bayesian Network, selecting the most flaky conflict pairs to guide specific thread interleaving, thereby more efficiently detecting flaky tests. Threats to Validity. In this study, we only perform one round of Bayesian Network evaluation on conflict pairs. The final experiment reveals that the conflict pairs with high flaky degree tend to exhibit a clustered distribution. Considering this characteristic, we plan to optimize the evaluation method for conflict pairs in the future. We will use the flaky values obtained from the current Bayesian Network as the prior results of the algorithm and adjust the probabilities based on the influence between conflict groups as the Bayesian Network posterior. This approach aims to better align with the analysis of clustered conflict pairs.

In our future endeavors, we aim to conduct experiments in larger software systems to further validate our findings and enhance the robustness of our methodology. Expanding the scale of our experiments will not only provide deeper insights into the dynamics of flaky tests but also facilitate comparative studies against other established tools designed for flaky test detection. This comparative analysis will enable us to identify optimization opportunities, refining our approach and contributing to the development of more effective strategies in this area. Additionally, we recognize the necessity of refining our experiments concerning scheduling points. Enhancing these experiments is crucial for validating the rationality of our thread scheduling methods. By meticulously examining the interplay between scheduling decisions and test outcomes, we aspire to derive a more nuanced understanding of how scheduling impacts the occurrence of flaky tests. This will ultimately lead to a more comprehensive framework for addressing the challenges posed by concurrency in software testing, thereby advancing the state of the art in flaky test detection and mitigation.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grant No. 62077003, the major project of teaching and research in Beijing University of Chemical Technology (2021BHDJGYB18).

REFERENCES

[1] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, pages 643–653, 2014.

[2] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. Understanding flaky tests: The developer's perspective. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 830–840, 2019.

[3] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. Analyzing inter-application communication in android. In *Proceedings of the 9th international conference on Mobile systems, applications, and services*, pages 239–252, 2011.

[4] Tanakorn Leesatapornwongsa, Xiang Ren, and Suman Nath. Flakerepro: Automated and efficient reproduction of concurrency-related flaky tests. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1509–1520, 2022.

[5] Martin Gruber, Muhammad Firhard Roslan, Owain Parry, Fabian Scharnböck, Phil McMinn, and Gordon Fraser. Do automatic test generation tools generate flaky tests? In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–12, 2024.

[6] Zhen Dong, Abhishek Tiwari, Xiao Liang Yu, and Abhik Roychoudhury. Flaky test detection in android via event order exploration. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 367–378, 2021.

[7] Jue Wang, Yanyan Jiang, Chang Xu, Qiwei Li, Tianxiao Gu, Jun Ma, Xiaoxing Ma, and Jian Lu. Aatt+: Effectively manifesting concurrency bugs in android apps. *Science of Computer Programming*, 163:1–18, 2018.

[8] Jue Wang, Yepang Liu, Chang Xu, Xiaoxing Ma, and Jian Lu. E-greendroid: effective energy inefficiency analysis for android applications. In *proceedings of the 8th Asia-Pacific Symposium on Internetware*, pages 71–80, 2016.

[9] Chang-Seo Park and Koushik Sen. Randomized active atomicity violation detection in concurrent programs. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of software engineering*, pages 135–145, 2008.

[10] Pallavi Joshi, Chang-Seo Park, Koushik Sen, and Mayur Naik. A randomized dynamic program analysis technique for detecting real deadlocks. *ACM Sigplan Notices*, 44(6):110–120, 2009.

[11] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. Learning from mistakes: a comprehensive study on real world concurrency bug characteristics. In *Proceedings of the 13th international conference on Architectural support for programming languages and operating systems*, pages 329–339, 2008.

[12] Jonas Imbsweiler, Maureen Ruesch, Hannes Weinreuter, Fernando Puente León, and Barbara Deml. Cooperation behaviour of road users in t-intersections during deadlock situations. *Transportation research part F: traffic psychology and behaviour*, 58:665–677, 2018.

[13] Jeff Huang. Scalable thread sharing analysis. In *Proceedings of the 38th International Conference on Software Engineering*, pages 1097–1108, 2016.

[14] Lili Bo, Shujuan Jiang, Junyan Qian, Rongcun Wang, and Yafei Yao. Performance evaluation of data race detection based on thread sharing analysis with different granularities: An empirical study. *IEEE Access*, 7:73819–73829, 2019.

[15] Dongjie Chen, Yanyan Jiang, Chang Xu, and Xiaoxing Ma. On interleaving space exploration of multi-threaded programs. *Frontiers of Computer Science*, 15:1–12, 2021.

Authorized licensed use limited to: QUAID E AZAM UNIVERSITY. Downloaded on July 20,2026 at 09:41:35 UTC from IEEE Xplore. Restrictions apply.